

Final document of Tower defense project

By: Nguyen Vinh Khang – 101682945 – 1st year

Aalto bachelor's Programme in Science and Technology

1. General description

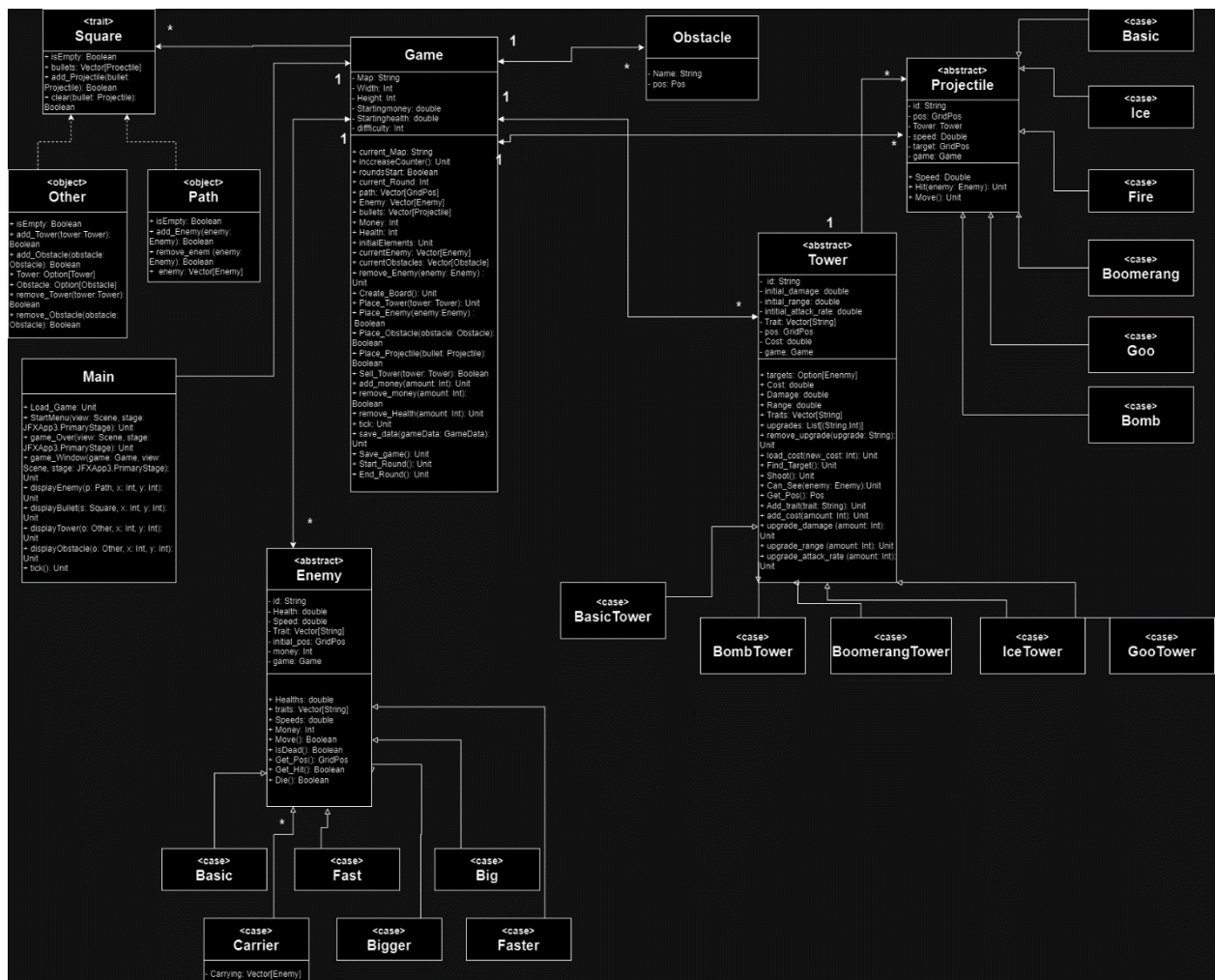
The game has many types of enemies in the game. Enemies have health points, speed and special characteristics depending on the type of the enemy. The special characteristics can be immunity to some damage, spawning more enemies, or invisibility to normal towers. The game also has many types of towers in the game. Towers have price, range, damage, attack rates and special characteristics. The special characteristics can be the ability to see invisible enemies, the ability to slow down enemies or different projectiles. Some of these special characteristics can be acquired by upgrading towers, and some will belong to certain towers. The tower can be upgraded to increase damage, attack rate, range or gain special characteristics. The tower can also be sold at 50% the original price which includes the cost of the upgrades. There can be obstacles that will block the vision of towers. The player can also change the speed of the game. The player can choose the difficulty of the game which will change the cost of the towers. The player can choose the color of the square. There are many maps in the game. The player can change the speed of the game. The player also has the ability to save their game then load it. I think the project is completed on the demanding level. I designed all of the maps and the player can not add their own maps to the program without changing the program code.

2. User interface

As the game is a tower defense game, all of the inputs from the player will be from the mouse. When you start the game, there will be a starting screen with three buttons, two color pickers and a note label. The first button allows you to create the new game. The second one allows you to load a game and the third one just closes the program. For the color pickers, they allow you to pick the color of the path square and other square. If you click the first button, it will lead you to another view with buttons representing difficulty. If you click any button, it will lead you to a new view with 5 images representing 5 maps for you to choose. When you choose a map, a new view will appear, and it is the game screen. There are 4 regions in the game screen: the game panel, the menu panel, the information panel and the upgrade panel. In the game panel, there will be two kinds of squares: the white one and the green one. The green square is the square which the enemies travel on and the white one is where we place our towers. There will be a red rectangle on one of the green squares which represents the end of the path. There will also be some black rectangles on the white squares which represent obstacles. In the information panel, there is a white box with texts, usually depicting information. In the menu panel, there are labels which show what round it is, how much health and money do you have. There are also many buttons

detailing the type of tower and its cost. There are also more buttons for other functions like starting the round, saving and quitting the game. There are also radio buttons that let you pick the speed of the game by clicking on the button at the speed you want. There will be nothing in the upgrade panel at the start. When you click one of the buttons that represent towers, you can hover the mouse on the game panel to see its effects. You can see the range of the tower if you place the tower there. When you want to place the tower, you have to click the square and it will place the tower. To start the round, you have to click the start button, and the game will begin spawning enemies. When your health gets to 0, the game will immediately switch to the game over screen. At the game over screen, you can retry the game and restart from round 1 or quit the game entirely.

3. Program structure



I divided the application into two main parts: the GUI and the Classes. I divide Classes part into smaller parts: Game, Enemy, Tower, Projectile, Obstacle, Square. The GUI depicts how the game looks, and the Game class will function as the combination of all components and serve as the gates that connect the GUI and the logic of the functions. I use Enemy, Tower, Obstacle, Projectile

classes to depict enemy, tower, obstacle and projectile respectively. The Square class is to depict the function of each square on the game board. As I use O1 library to create the game board, the Square class is essential. The Game class has multiple Towers, Enemies, Obstacles and Projectile, and it also can access the information in each instance of those classes, as the Game class has all of the information about the current state of the game. In addition, each instance of those classes can access the information in the game class. For the user interface, the Main.scala file depicts the GUI while the Game class depicts the logic components behind the game. In the Main.scala, there are many functions that depict different views, and the game will change to different view when called.

4. Algorithms

Most of the algorithms for the solution methods are very simple. I will explain the more complex algorithms like enemy moving, the trajectory of projectiles, game ticking and tower shooting.

For the enemy to move, I use a list to store the grid position of the path where the enemy needs to go. In this way, the enemy does not need to compute anything, and it only has to traverse the elements of the list. So, the enemy will go where I set it to go. However, I have to use a different algorithm for the movement of projectiles. As I use O1 Grid class to create the game board, it is very difficult to simulate the diagonal path to the projectile's target. There are two types of movements of projectiles: the normal movement and the boomerang movement. For the boomerang movement, the projectiles' path will look like a rectangle. It will start from the tower and go to the target position and return to the tower. The projectiles will first go horizontally to close the gap between their position and the target position by one square each time they move until they reach the same horizontal position of their targets, and they will then go vertically to the target position the same way. Finally, it will repeat the process by replacing the target position with the original tower's position. From the description, I create a Boolean variable to check whether the projectiles have reached the target position and I also calculate the horizontal and vertical difference between the projectile's position and the target's position to determine which way it will move. If the projectile hits an obstacle, the projectile will disappear. For the normal movement, I first calculate the horizontal and vertical differences of the projectile's position and the target's position. If both the horizontal and vertical differences are larger or equal to 2, I will determine whether the horizontal difference or the vertical difference is larger. If the horizontal difference is larger, the projectile will move 2 squares closer horizontally and 1 square closer vertically. If the vertical difference is larger, the projectile will move 2 squares closer vertically, 1 square closer horizontally. If both are equal, the projectile will move 2 squares closer vertically and 2 squares closer horizontally. However, the projectile will move straight to the target's position if either the horizontal or the vertical difference is less than 2. I also determine whether there are obstacles between the original position and the new position to remove the projectile.

For the tower to shoot, the tower needs to choose a target. Every time the game calls for the tower to shoot which is determined by the attack rate of the tower, the tower will calculate the distance between the enemies that are in the tower's range and it, and it will choose the closest enemy to be the target. I also account for the traits of the enemy and tower to determine whether the tower can see the enemy. After the tower has a target, the tower will spawn the correct projectile at the tower's position.

To start the round, I will have to choose a set of enemies randomly and spawn them one by one. First, I will calculate the difficulty number of the round by multiplying the round the number by 5. Then I will generate random numbers and add it to a list so that all of the numbers in the list add up to the difficulty number. From the list, I will create another list with enemies based on the list of numbers. One number will correspond to one type of enemy. From the whole list of enemies, I will create another list that will contain the current enemies that are on the screen. When the game ticks 10 times, it will add one enemy to the current enemies list and spawn it. After that, the game will keep ticking until the round ends which is when all of the enemies and projectiles disappear. When the game ticks, there will be a counter that will determine when a function is called. For example, an enemy will only move when the game ticks the same amount of time as the speed of the enemy. In this way, I can prevent the enemies, towers and projectiles' function from being called every time the game ticks.

5. Data structures

I mostly used existing Scala structures such as List, Vector, ... The only structure I created was for the save format. I created a new structure so I can access the information in the save file easier, and it also made the saving process become simpler. The SaveData structure consists of the round number, money, health, map, difficulty and a list of towers as strings. From the save data, I can write to the save file as a json file using upickle. I also use the Grid class in O1 to create the game board. The Grid class represents a rectangular grid that contains elements that are all one type. The Grid class allows me to create the game board easily and it also helps make calculation simpler as I have the coordinates of the grids. Most of the arrays I use are mutable because the game needs to update the information constantly, so the arrays need to be mutable to be able to change on the fly. The arrays are used to store most important information as the information usually consists of multiple elements such as enemies, towers and projectiles.

6. Files and internet access

The game does not have the ability to access the Internet. The game only deals with json files, and it only deals with one file which is the saveData.json in the resources directory. The player does not have to make their own save file. The game will have a default save file which is saveData.json. In the save data, there will be numbers for round, health, money, difficult. There will be a string for the map and a list of string representing towers. When you load the save file, the game will read the save file and remember the variables and make a new instance of the Game class with the new variables. I want the game to remember the upgrades of each tower,

but towers can have different upgrades, and it can make the saving process more complicated. So, I decided to make the starting damage, range and attack rate of each type of towers constant to determine if the player has upgraded the towers. For example, the file will look like the example below.

```
{
  "data": [
    {
      "Round": 2,
      "Money": 80,
      "Health": 7,
      "Map": "Map5",
      "Difficulty": 2,
      "Towers": [
        "Basic_Tower, (10,5), 5, 3.0, 15.0, 50",
        "Basic_Tower, (9,2), 5, 3.0, 15.0, 50"
      ]
    }
  ]
}
```

7. Testing

For the complex functions, I use unit testing to test each function. In this way, I can make sure all of the components work as intended before putting the functions in a bigger function such as the tick function in the Game class. For the tick function and other elements that require GUI such as saving and loading save file, I use the graphical user interface to test them. It is easier to test them in this way because the functions' effect can be seen clearly through the GUI. From there, I can catch bugs and fix them. The program passed all of the unit tests I designed and functioned correctly in the GUI.

8. Known bugs and missing features.

I have fixed all of the bugs I could find, so everything works correctly as far as I know. As for missing features, I have decided to not implement the timer for the game as I do not think it will

contribute greatly to the gameplay, and I also have not implemented the removing obstacle function.

9. Strengths and weaknesses

For the weaknesses, I think the error handling of my game is quite bad. I prevent most of the errors from happening in the first place by limiting the player to what the player can do. However, I do not provide any error messages, or the error message is not very helpful. Another weakness is the balance of the game is not great. Some towers are too strong that makes other towers seem redundant. Another weakness is the unit tests of the program only covers the basic examples. There may be bugs that exist in another test example. I only use the unit tests to ensure that it works as intended and then I find bugs with the GUI.

For the strengths, I think that the program has many functions that allow the player to interact and configure the game such as changing the game speed and upgrading towers. It makes the gameplay more interactive. Another strength may be the GUI. Although the GUI is simple, it works very well, and it is quite easy to learn how the GUI functions. Another strength may be the program structures. The program structures are divided into clear classes that are all combined in a large class. This makes it easier for me to build from the bottom to the top. It also allows me to know in which class the bugs are located, and it becomes easier for me to fix bugs.

10. Schedules

I believe I am on schedule every two weeks. For the first two weeks, I implemented most of the functions in the classes. I implemented the trait Square, the Tower, Enemy, Projectile class and most of the Game class. For the next two weeks, I implemented the GUI and connected it to the logic of the game, and I also implemented the tick function to spawn enemies. For the next two weeks, I implemented saving and loading functions and added unit tests. I also implemented upgrading and selling towers. For the final two weeks, I improved the GUI, fixed bugs and added more unit tests. For the first and second two weeks, it takes me longer to complete all of my plans, about 5 – 10 hours, while it takes me shorter time to complete my plans for the third and final two weeks. I believe that it is because the workload in the first and second weeks are very heavy, and I underestimated how long it would take. I follow the plan closely, as I do not plan forward, and I just planned my workload for the next two weeks when I finished all my tasks. From the projects, I learned to make my plans more realistic and divide the workload more balanced.

11. Final evaluation

Finally, evaluate what you would do differently if you started the project again from the beginning.

I think the quality of the final program is good. It's very great, but it is sufficient. I think that the GUI is quite simple, and I could improve it more. As mentioned before, the error handling and the unit tests of the game is not very good. I could have improved them better in the future. For now, I think that the data structures and class structure are good. It can be tweaked and improved to

implement more functions such as removing obstacles. I think that most of the solution methods are good. However, I feel like the trajectory of the projectiles can be improved to simulate the diagonal movement better. For now, I do not know how to improve it, but I think it is possible. The structure of the program is suitable for making changes, as many of the class structure can be easily tweaked to add more functions. If I started the project from the beginning, I would divide my workload more balanced, and that could improve the quality of the project. I could implement the functions that I left out and improve some of the solution methods.

12. References

O1 scala docs and API: used to create the game board

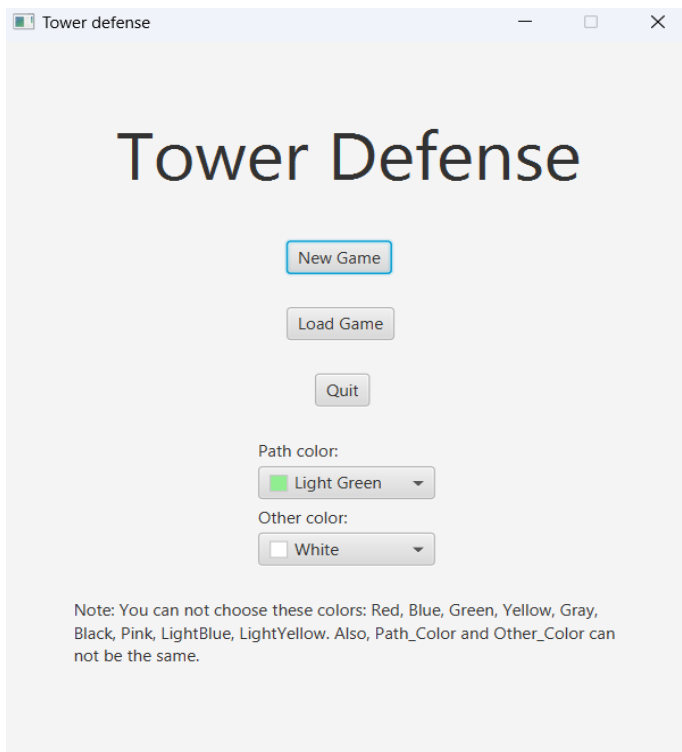
https://gitmanager.cs.aalto.fi/static/O1_2022/modules/given/O1Library/doc/docs/credits.html

ScalaFX API: used for the GUI

https://javadoc.io/doc/org.scalafx/scalafx_2.13/latest/index.html

13. Appenndix

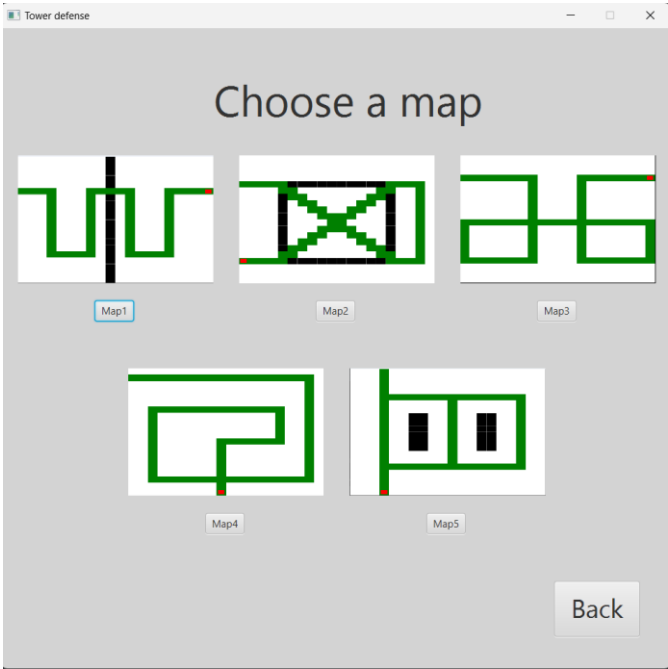
The full source code will be included as an IntelliJ project in the final submission.



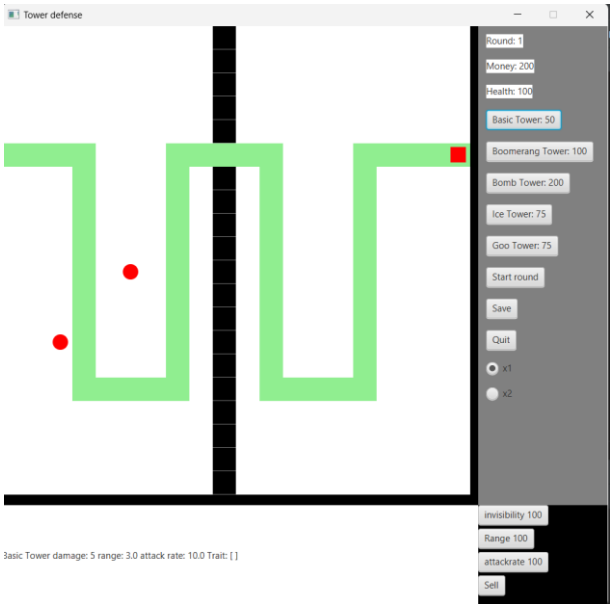
Start menu screen



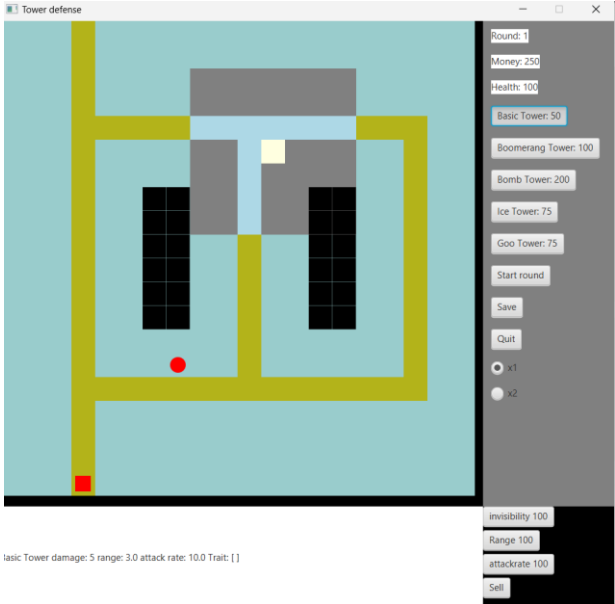
Difficulty screen



Map choosing screen



Game window



game window with different map
And color with range indicator